

**INSTITUTO
FEDERAL**
Santa Catarina

Funções e Métodos de Arrays

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas

cores

#111027

#277756

#16ABCD

#FFF4EC

#C74E23

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono



Coleções (Array)

Verificando Arrays em JavaScript

- **Array.isArray()** é um método útil para determinar se um valor é um array.

Coleções (Array)

Verificando Arrays em JavaScript

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;

Coleções (Array)

Verificando Arrays em JavaScript

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?

Coleções (Array)

Verificando Arrays em JavaScript

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.



Coleções (Array)

Verificando Arrays em JavaScript

exemplos

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.

```
Array.isArray([1, 2, 3]);
```



Coleções (Array)

Verificando Arrays em JavaScript

exemplos

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.

```
Array.isArray([1, 2, 3]); // true
```



Coleções (Array)

Verificando Arrays em JavaScript

exemplos

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.

```
Array.isArray([1, 2, 3]); // true  
Array.isArray("texto");
```



Coleções (Array)

Verificando Arrays em JavaScript

exemplos

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.

```
Array.isArray([1, 2, 3]); // true  
Array.isArray("texto"); //false
```



Coleções (Array)

Verificando Arrays em JavaScript

exemplos

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.

```
Array.isArray([1, 2, 3]); // true  
Array.isArray("texto"); //false  
Array.isArray({ chave: "valor" });
```



Coleções (Array)

Verificando Arrays em JavaScript

exemplos

- **Array.isArray()** é um método útil para determinar se um valor é um array.
 - `Array.isArray(valor);`
 - Retorna true ou false;
 - Por que usar?
 - Evita erros ao verificar tipos de variáveis.
 - Útil quando recebemos dados dinâmicos.

```
Array.isArray([1, 2, 3]); // true
Array.isArray("texto"); //false
Array.isArray({ chave: "valor" }); //false
```



Coleções (Array)

métodos de arrays (classe Array)

Coleções (Array)

métodos de arrays (classe Array)

- bastante poderosos

Coleções (Array)

métodos de arrays (classe Array)

- bastante poderosos
- **podem** modificar o array a partir do qual são invocados

Coleções (Array)

métodos de arrays (classe Array)

- bastante poderosos
- **podem** modificar o array a partir do qual são invocados
- **alguns** retornam um array:
 - retornam um novo array; original permanece
 - modificam o original; retornam sua referência (como o novo tamanho ou o elemento removido).

Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
 - invoca a função com 3 argumentos:

exemplos



Coleções (Array)

métodos de arrays (classe Array)

exemplos

- **forEach()** invoca uma função para cada elemento do array
 - invoca a função com 3 argumentos:
 1. o valor do elemento
 2. o índice do elemento
 3. o próprio array

Coleções (Array)

métodos de arrays (classe Array)

exemplos

- **forEach()** invoca uma função para cada elemento do array
 - invoca a função com 3 argumentos:
 1. o valor do elemento
 2. o índice do elemento
 3. o próprio array
 - não é possível interromper a iteração (break)

Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
 - invoca a função com 3 argumentos:
 1. o valor do elemento
 2. o índice do elemento
 3. o próprio array
 - não é possível interromper a iteração (break)

exemplos

```
let data = [1, 2, 3, 4, 5], soma = 0;
data.forEach(valor => { soma += valor; });

data.forEach(function (v, i, a)
    { a[i] = v + 1; }
);
```

Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
 - a função deve retornar um valor
 - o retorno é um novo array

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
 - a função deve retornar um valor
 - o retorno é um novo array

exemplos

```
let a = [1, 2, 3];  
let quad = a.map(x => x * x);
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
 - a função deve retornar um valor
 - o retorno é um novo array

exemplos

```
let a = [1, 2, 3];  
let quad = a.map(x => x * x); // [1, 4, 9]
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro

exemplos

Coleções (Array)

métodos de arrays (classe Array)

exemplos

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
 - a função filtro deve retornar um booleano (true ou false)
 - se o valor retornado for true, o elemento é incluído no array de retorno



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
 - a função filtro deve retornar um booleano (true ou false)
 - se o valor retornado for true, o elemento é incluído no array de retorno

exemplos

```
let a = [5, 4, 3, 2, 1];  
  
a.filter(x => x < 3); // [2, 1];
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
 - a função filtro deve retornar um booleano (true ou false)
 - se o valor retornado for true, o elemento é incluído no array de retorno

exemplos

```
let a = [5, 4, 3, 2, 1];  
  
a.filter(x => x < 3); // [2, 1];  
  
a.filter( (x,i) => i % 2 === 0 );  
// [5, 3, 1];
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true

exemplos



Coleções (Array)

métodos de arrays (classe Array)

exemplos

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3);
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```

```
a.findIndex(x => x < 0);
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```

```
a.findIndex(x => x < 0); // -1
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```

```
a.findIndex(x => x < 0); // -1
```

```
a.find(x => x % 5 === 0);
```

Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```

```
a.findIndex(x => x < 0); // -1
```

```
a.find(x => x % 5 === 0); // 5
```



Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```

```
a.findIndex(x => x < 0); // -1
```

```
a.find(x => x % 5 === 0); // 5
```

```
a.find(x => x % 7 === 0);
```

Coleções (Array)

métodos de arrays (classe Array)

- **forEach()** invoca uma função para cada elemento do array
- **map()** passa cada elemento para a função invocada e retorna um array com os resultados da função
- **filter()** retorna um subconjunto do array baseado na função filtro
- **find(); findIndex()** similar a **filter()**, mas param no primeiro true
 - **find()** retorna o elemento
 - **findIndex()** retorna o índice

exemplos

```
let a = [1,2,3,4,5];
```

```
a.findIndex(x => x === 3); // 2
```

```
a.findIndex(x => x < 0); // -1
```

```
a.find(x => x % 5 === 0); // 5
```

```
a.find(x => x % 7 === 0); // undefined
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays

exemplos

Coleções (Array)

métodos de arrays (classe Array)

exemplos

- **concat()** concatena arrays
 - retorna um **novo** array contendo os elementos do original e os argumentos

Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
 - retorna um **novo** array contendo os elementos do original e os argumentos

exemplos

```
let a = [1,2,3];
```

```
a.concat(4, 5); // [1,2,3,4,5]
```

Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
 - retorna um **novo** array contendo os elementos do original e os argumentos
 - se algum argumento for um array apenas os elementos são concatenados

exemplos

```
let a = [1,2,3];
```

```
a.concat(4, 5); // [1,2,3,4,5]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
 - retorna um **novo** array contendo os elementos do original e os argumentos
 - se algum argumento for um array apenas os elementos são concatenados

exemplos

```
let a = [1,2,3];
```

```
a.concat(4, 5); // [1,2,3,4,5]
```

```
a.concat([6,7],[8,9]); // [1,2,3,6,7,8,9]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
 - retorna um **novo** array contendo os elementos do original e os argumentos
 - se algum argumento for um array apenas os elementos são concatenados
 - mas **não** array aninhados!

exemplos

```
let a = [1,2,3];
```

```
a.concat(4, 5); // [1,2,3,4,5]
```

```
a.concat([6,7],[8,9]); // [1,2,3,6,7,8,9]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
 - retorna um **novo** array contendo os elementos do original e os argumentos
 - se algum argumento for um array apenas os elementos são concatenados
 - mas **não** array aninhados!

exemplos

```
let a = [1,2,3];
```

```
a.concat(4, 5); // [1,2,3,4,5]
```

```
a.concat([6,7],[8,9]); // [1,2,3,6,7,8,9]
```

```
a.concat([10,[11,12]]); // [1,2,3,10,[11,12]]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (inclusivo)
 - segundo parâmetro define o fim do subconjunto (não inclusivo)
 - não modifica o array original

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (inclusivo)
 - segundo parâmetro define o fim do subconjunto (não inclusivo)
 - não modifica o array original

exemplos

```
let a = [1, 2, 3, 4, 5];
```

```
a.slice(0, 3);
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (inclusivo)
 - segundo parâmetro define o fim do subconjunto (não inclusivo)
 - não modifica o array original

exemplos

```
let a = [1, 2, 3, 4, 5];
```

```
a.slice(0, 3); // [1,2,3]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (inclusivo)
 - segundo parâmetro define o fim do subconjunto (não inclusivo)
 - não modifica o array original

exemplos

```
let a = [1, 2, 3, 4, 5];
```

```
a.slice(0, 3); // [1,2,3]
```

```
a.slice(3); // [4,5]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (inclusivo)
 - segundo parâmetro define o fim do subconjunto (não inclusivo)
 - não modifica o array original

exemplos

```
let a = [1, 2, 3, 4, 5];
```

```
a.slice(0, 3); // [1,2,3]
```

```
a.slice(3); // [4,5]
```

```
a.slice(1, -1); // [2,3,4]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (inclusivo)
 - segundo parâmetro define o fim do subconjunto (não inclusivo)
 - não modifica o array original

exemplos

```
let a = [1, 2, 3, 4, 5];  
  
a.slice(0, 3); // [1,2,3]  
a.slice(3); // [4,5]  
a.slice(1, -1); // [2,3,4]  
a.slice(-3, -2);
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
 - primeiro parâmetro define o início do subconjunto (**inclusivo**)
 - segundo parâmetro define o fim do subconjunto (**não inclusivo**)
 - **não** modifica o array original

exemplos

```
let a = [1, 2, 3, 4, 5];  
  
a.slice(0, 3); // [1,2,3]  
a.slice(3); // [4,5]  
a.slice(1, -1); // [2,3,4]  
a.slice(-3, -2); // [3]
```

Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)

exemplos



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)

exemplos

```
let a = ["banana", "maçã", "açaí"];  
a.sort(); // a == ["açaí", "banana", "maçã"]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)

exemplos

```
let a = ["banana", "maçã", "açaí"];  
a.sort(); // a == ["açaí", "banana", "maçã"]
```

```
let b = [33, 4, 1111, 222];  
b.sort();
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)

exemplos

```
let a = ["banana", "maçã", "açaí"];  
a.sort(); // a == ["açaí", "banana", "maçã"]
```

```
let b = [33, 4, 1111, 222];  
b.sort(); // b == [1111, 222, 33, 4]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)
 - **com argumento** função de ordenação

exemplos

```
let a = ["banana", "maçã", "açaí"];  
a.sort(); // a == ["açaí", "banana", "maçã"]
```

```
let b = [33, 4, 1111, 222];  
b.sort(); // b == [1111, 222, 33, 4]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)
 - **com argumento** função de ordenação
 - **< 0** primeiro argumento antes
 - **== 0** indiferente
 - **> 0** primeiro argumento depois

exemplos

```
let a = ["banana", "maçã", "açaí"];  
a.sort(); // a == ["açaí", "banana", "maçã"]
```

```
let b = [33, 4, 1111, 222];  
b.sort(); // b == [1111, 222, 33, 4]
```



Coleções (Array)

métodos de arrays (classe Array)

- **concat()** concatena arrays
- **slice()** retorna um subconjunto do array
- **sort()** ordena os elementos no próprio array e retorna o array ordenado
 - **sem argumentos** ordem "alfabética" (unicode)
 - **com argumento** função de ordenação
 - **< 0** primeiro argumento antes
 - **== 0** indiferente
 - **> 0** primeiro argumento depois

exemplos

```
let a = ["banana", "maçã", "açaí"];  
a.sort(); // a == ["açaí", "banana", "maçã"]
```

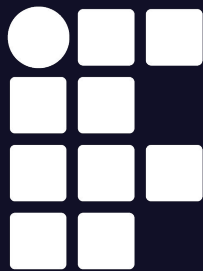
```
let b = [33, 4, 1111, 222];  
b.sort(); // b == [1111, 222, 33, 4]
```

```
b.sort((a,b) => {return a - b;});  
[3,2,1].sort(function(a,b){return a - b;});  
[10, 2, 30].sort((a, b) => a - b); // [2, 10, 30]  
[10, 2, 30].sort((a, b) => b - a); // [30, 10, 2]
```



Exercícios

Resolva a lista de exercícios do SIGAA



**INSTITUTO
FEDERAL**
Santa Catarina

Funções e Métodos de Arrays

Programação Frontend I

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

PENSAMENTO
COMPUTACIONAL E
ALGORITMOS

Análise e Desenvolvimento
de Sistemas